

Trabalho Prático I - AEDS III: Manipulação de Arquivo Sequencial e Ordenação Externa

Bruno Pais Moacir

¹Instituto de Ciências Exatas e Informática (ICEI)
Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Belo Horizonte – MG – Brasil

`bruno@sga.pucminas.br`

Abstract. *This report details the implementation of a sequential file management system developed for the Data Structures and Algorithms III (AEDS III) course. The project involves parsing a public domain dataset, performing CRUD operations directly on a binary file using tombstone logic for logical deletion, and executing an external sorting algorithm (Balanced Multiway Merging). The core constraints included avoiding high-level native string manipulation methods, ensuring structural robustness, and managing variable-length records in secondary memory.*

Resumo. *Este relatório detalha a implementação de um sistema de gerenciamento de arquivos sequenciais desenvolvido para a disciplina de Algoritmos e Estruturas de Dados III (AEDS III). O projeto envolve a leitura de uma base de dados de domínio público, a execução de operações CRUD diretamente em um arquivo binário utilizando a lógica de lápides para exclusão lógica, e a execução de um algoritmo de ordenação externa (Intercalação Balanceada). As principais restrições incluíram evitar métodos nativos de alto nível para manipulação de strings, garantir a robustez estrutural e gerenciar registros de tamanho variável em memória secundária.*

1. Introdução

O Trabalho Prático I de Algoritmos e Estruturas de Dados III tem como objetivo consolidar os conceitos de armazenamento de dados em memória secundária. A manipulação de arquivos sequenciais requer uma gestão rigorosa do espaço, especialmente quando os registros possuem tamanhos variados. Para este projeto, foi selecionada a base de dados *IMDb Movies*, contemplando campos obrigatórios como strings de tamanho variável (título do filme), strings de tamanho fixo (sigla do país), datas de lançamento, listas de valores (gêneros) e valores de ponto flutuante (avaliações).

2. Desenvolvimento

O sistema foi desenvolvido integralmente em Java, com forte ênfase na clareza de codificação e obediência às restrições arquiteturais propostas.

2.1. Carga e Parse Manual de Dados

A importação da base de dados CSV para o arquivo binário foi construída de forma estrita. Para evitar violações das regras da disciplina, bibliotecas ou métodos nativos complexos como `.split()` e `.replaceAll()` foram descartados. A separação de colunas, a

extração da lista de gêneros e a conversão de datas foram programadas manualmente utilizando laços de repetição e o método `.charAt()`, garantindo total controle sobre a varredura das strings.

2.2. Estrutura do Arquivo Binário e CRUD

A memória secundária (`dados.bin`) foi estruturada com um cabeçalho inicial de 4 bytes contendo o último ID gerado. Cada registro gravado sequencialmente obedece à estrutura: Lápide (1 byte), Indicador de Tamanho (4 bytes) e o Vetor de Bytes do objeto. As operações de CRUD lidam com a movimentação do ponteiro via `RandomAccessFile`. Destaca-se a operação de Atualização (*Update*): caso o novo vetor de bytes exceda o tamanho do registro original, o registro antigo recebe a marcação de exclusão lógica (lápide com `*`) e o registro atualizado é movido para o fim do arquivo.

2.3. Ordenação Externa

Para lidar com a desordenação causada pelos *updates* e para recuperar o espaço dos registros deletados logicamente, foi implementado o algoritmo de Intercalação Balanceada. O processo foi dividido em duas fases:

- **Distribuição:** O arquivo original é lido em blocos (limitados pela capacidade da memória primária). Os registros válidos são ordenados por ID e gravados em arquivos temporários (caminhos).
- **Intercalação:** Os arquivos temporários são lidos simultaneamente. O menor registro entre os caminhos é selecionado e gravado no arquivo final definitivo, garantindo a ordenação completa e a limpeza dos fragmentos deletados.

3. Testes e Resultados

Os testes foram conduzidos via terminal de linha de comando no ambiente Linux. A carga inicial gravou mais de 10.000 registros com sucesso. Operações de leitura retornaram os objetos formatados instantaneamente. Testes críticos de atualização provaram que o sistema gerencia corretamente o crescimento do vetor de bytes, validado pela busca imediata do registro após a exclusão do seu espaço original. Por fim, a Ordenação Externa foi executada com sucesso utilizando 3 caminhos e um limite de 20.000 registros, gerando um arquivo limpo e perfeitamente ordenado, sem corromper a integridade dos dados.

4. Conclusão

A implementação atingiu 100% de conformidade com as especificações. A construção manual do *parser* de strings e a estruturação em baixo nível do arquivo binário garantiram a robustez do programa. A correta execução da ordenação externa comprovou a eficiência da arquitetura escolhida para a gestão de memória secundária, preparando a base para as futuras implementações de indexação e árvores B+ nas próximas etapas da disciplina.